Installazione di JMH: Maven e Gradle

Versione usata: **JMH 1.37**

1. Maven

1.1 Struttura e dipendenze

Metti i benchmark in un modulo separato (consiglio), ad esempio benchmarks/, che dipende dal modulo contenente il codice da testare. In un progetto multi-modulo è solo il modulo (o i moduli) che vuoi misurare, non tutti gli altri. La struttura diventa:

```
my-project/
    pom.xml                  <- pom padre (radice)
    my-module/               <- modulo con il codice da testare (già
esistente)
        src/main/java/...
    benchmarks/              <- modulo nuovo
        pom.xml
        src/main/java/MyBenchmark.java
```

Nel pom.xml padre (radice) dichiara il nuovo modulo:

```
<modules>
    <module>my-module</module>
    <module>benchmarks</module>
</modules>
```

Nel benchmarks/pom.xml (nuovo) aggiungi il parent, la dipendenza verso il modulo da testare e le dipendenze JMH (sostituisci my-project, my-module e com.example con i valori del tuo progetto):

```
<parent>
    <groupId>com.example</groupId>
    <artifactId>my-project</artifactId>
    <version>1.0</version>
</parent>

<artifactId>benchmarks</artifactId>

<dependencies>
    <!-- modulo che contiene il codice da testare -->
    <dependency>
        <groupId>com.example</groupId>
        <artifactId>my-module</artifactId>
        <version>${project.version}</version>
    </dependency>
    <dependency>
```

```
        <groupId>org.openjdk.jmh</groupId>
        <artifactId>jmh-core</artifactId>
        <version>1.37</version>
    </dependency>
    <dependency>
        <groupId>org.openjdk.jmh</groupId>
        <artifactId>jmh-generator-annprocess</artifactId>
        <version>1.37</version>
        <scope>provided</scope>
    </dependency>
</dependencies>
```

1.2 Plugin per creare il jar eseguibile (in benchmarks/pom.xml)

```
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-shade-plugin</artifactId>
            <version>3.5.3</version>
            <executions>
                <execution>
                    <phase>package</phase>
                    <goals>
                        <goal>shade</goal>
                    </goals>
                    <configuration>
                        <finalName>benchmarks</finalName>
                        <transformers>
                            <transformer
implementation="org.apache.maven.plugins.shade.resource.ManifestResour
ceTransformer">

<mainClass>org.openjdk.jmh.Main</mainClass>
                            </transformer>
                        </transformers>
                    </configuration>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>
```

Nota: con JDK 23 o superiore l'annotation processing implicito è disattivato. In quel caso aggiungi, nel benchmarks/pom.xml, a maven-compiler-plugin un blocco annotationProcessorPaths con jmh-generator-annprocess.

1.3 Benchmark

Crea la classe nel nuovo modulo, in benchmarks/src/main/java, ad esempio benchmarks/src/main/java/MyBenchmark.java:

```
import org.openjdk.jmh.annotations.Benchmark;

public class MyBenchmark {

    @Benchmark
    public int test() {
        return "hello".length();
    }
}
```

1.4 Esecuzione

```
mvn clean package
java -jar benchmarks/target/benchmarks.jar
```

Da eseguire dalla radice del progetto: il jar viene creato in benchmarks/target/.
Usando le opportune opzioni, tramite il jar puoi scegliere il preciso microbenchmark da eseguire e la sua configurazione (o li puoi eseguire tutti con una specifica configurazione).

---

2. Gradle

2.1 Plugin

Nel build.gradle.kts **del modulo che contiene il codice da testare**:

```
plugins {
    java
    id("me.champeau.jmh") version "0.7.3"
}

repositories {
    mavenCentral()
}

jmh {
    jmhVersion = "1.37"
}
```

Equivalente in build.gradle (Groovy):

```
plugins {
    id 'java'
    id 'me.champeau.jmh' version '0.7.3'
}

repositories {
    mavenCentral()
```

```
}

jmh {
    jmhVersion = '1.37'
}
```

2.2 Benchmark

Crea la classe nella cartella src/jmh/java del modulo che contiene il codice da testare, ad esempio src/jmh/java/MyBenchmark.java. I file in src/jmh/java vedono le classi di src/main dello stesso modulo; le eventuali risorse vanno in src/jmh/resources:

```
import org.openjdk.jmh.annotations.Benchmark;

public class MyBenchmark {

    @Benchmark
    public int test() {
        return "hello".length();
    }
}
```

2.3 Esecuzione

```
./gradlew jmh
```

I risultati vengono salvati in build/results/jmh/. Con più moduli, il plugin va applicato nel modulo che contiene il codice da testare e il task si lancia su quel modulo: ./gradlew :nome-modulo:jmh (risultati in nome-modulo/build/results/jmh/).
Anche qui è possibile usare le istruzioni da riga di comando per selezionare precisi microbenchmark e decidere configurazioni.